Integrating Adobe Experience Manager (AEM) with React can be accomplished through two primary architectural approaches: the **AEM SPA Editor (hybrid/coupled approach)** and the **Headless CMS approach (decoupled approach)**. The best choice depends on specific project requirements, particularly regarding content authoring experience and frontend control.

1. AEM SPA Editor (Hybrid/Coupled)

This approach leverages the AEM SPA Editor SDK to allow authors to create and edit content within AEM's familiar in-context editing interface (WYSIWYG) while the frontend is built entirely with React components.

- **How it works:** AEM manages the page structure and content data as a JSON model. The React application consumes this JSON and dynamically renders the components. The AEM editor uses a JavaScript SDK to synchronize the editing experience with the React app.
- **Best for:** Projects where content authors need full control over layout and content placement using AEM's drag-and-drop features.
- **Setup:** Use the AEM Project Archetype and set the `frontendModule` property to `react` during project generation to bootstrap a starter project with the necessary configurations.

2. Headless CMS (Decoupled)

In this approach, AEM acts purely as a content repository and delivery service. The React application is a separate, external entity that fetches content from AEM via APIs.

- **How it works:** Content is typically structured using **Content Fragments** in AEM. The React app uses the AEM Headless SDK or GraphQL APIs to query and retrieve the required content in JSON format. The frontend team has complete control over the presentation layer.
- **Best for:** Mobile apps, single-page applications (SPAs) with complex user experiences, or when a complete separation of concerns between frontend and backend is desired.
- **Setup:** The React app can be built using standard tooling (like Create React App). You configure the app to connect to AEM's GraphQL endpoint using environment variables and utilize the AEM Headless Client for JavaScript to fetch data.

Summary of Key Differences

Feature	AEM SPA Editor (Hybrid)	Headless CMS (Decoupled)
Authoring	In-context WYSIWYG editing in AEM	Content managed in AEM, previewed in the React app (potentially using Universal Editor)

Frontend Control	Shared control; dependent on AEM SDK	Full control with any React tooling/framework
Deployment	Tightly coupled with AEM deployment cycle	Independent frontend deployment, potentially using AEM front-end pipelines
APIs Used	Page Model Manager (JSON output)	GraphQL APIs or Content Services

Choosing the Right Approach

- **Use the AEM SPA Editor** if you need AEM authors to have full control over the page layout and components using AEM's authoring interface.
- **Use the Headless approach** if you are building a pure React application (e.g., a mobile app, or a site built with Next.js) that simply needs content from AEM, and the authors can work with structured content fragments.